



IF3270 Pembelajaran Mesin

Recurrent Neural Network (ANN)

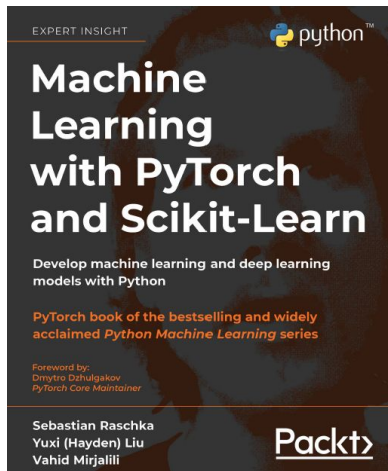
Tim Pengajar IF3270



Review

- Machine learning (ML) overview
- Ensemble Methods → Supervised Learning
- Supervised Learning: Perceptron
- Supervised Learning: ANN: Feed Forward Neural Network
- Supervised Learning: ANN: Convolutional Neural Network
- Supervised Learning: ANN: Recurrent Neural Network → Today

References

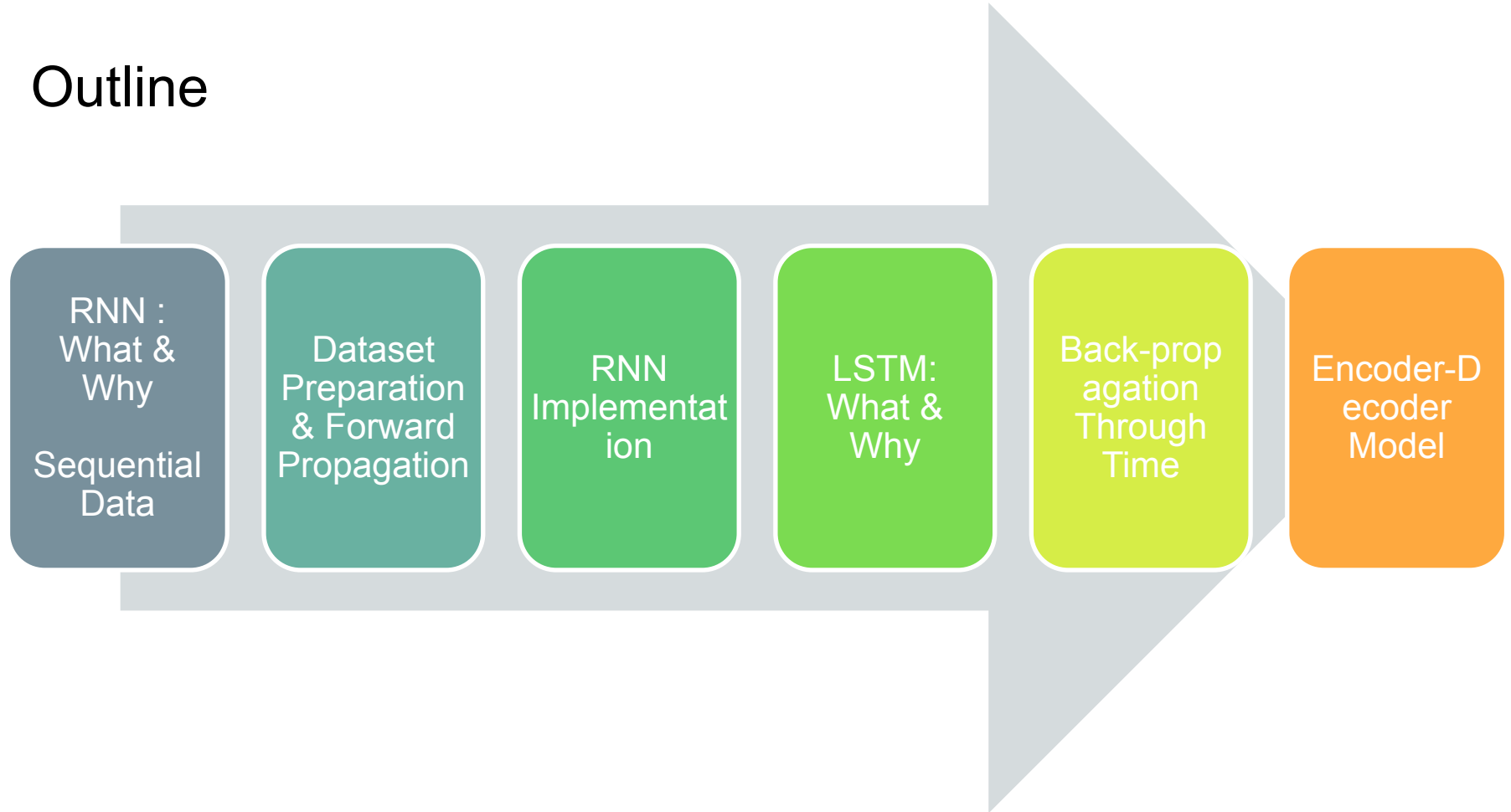


Raschka, et.al., Machine Learning with Pytorch and Scikit-Learn, Packt Publishing Ltd., 2022 (Chapter 15)

Chapter 15: Modeling Sequential Data Using Recurrent Neural Networks

- [-] Introducing sequential data
 - Modeling sequential data – order matters
 - Sequential data versus time series data
 - Representing sequences
 - The different categories of sequence modeling
- [-] RNNs for modeling sequences
 - Understanding the dataflow in RNNs
 - Computing activations in an RNN
 - Hidden recurrence versus output recurrence
 - The challenges of learning long-range interactions
 - Long short-term memory cells
- [+] Implementing RNNs for sequence modeling in PyTorch
- Summary

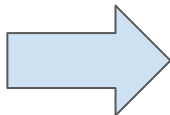
Outline



Sequential Data (aka Sequence Data or Sequences)

Assumption of supervised learning: **independent and identically distributed (IID)** data

- mutual independence assumption, **the order** in which the training examples are given to the model is **irrelevant**
- models **do not** have a memory of **previously seen** training examples



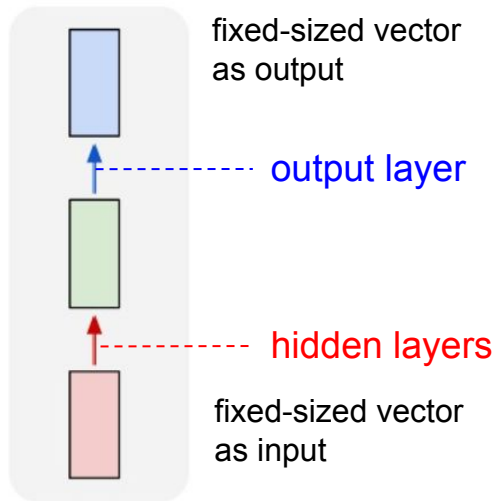
this assumption is **not valid** when we deal with sequences, **order matters**

Time dimension determines the **order** among the data points. Time series data is a special type of sequential data where each example is associated with a dimension for time. For example, stock prices and voice or speech records.

Not all sequential data has the time dimension. For example, in text data or DNA sequences, the examples are ordered, but text or DNA does not qualify as time series data.

Vanilla Neural Network

one to one



Each rectangle is a vector and arrows represent functions (e.g. matrix multiply). Input vectors are in red, output vectors are in blue and green vectors hold the hidden state.

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

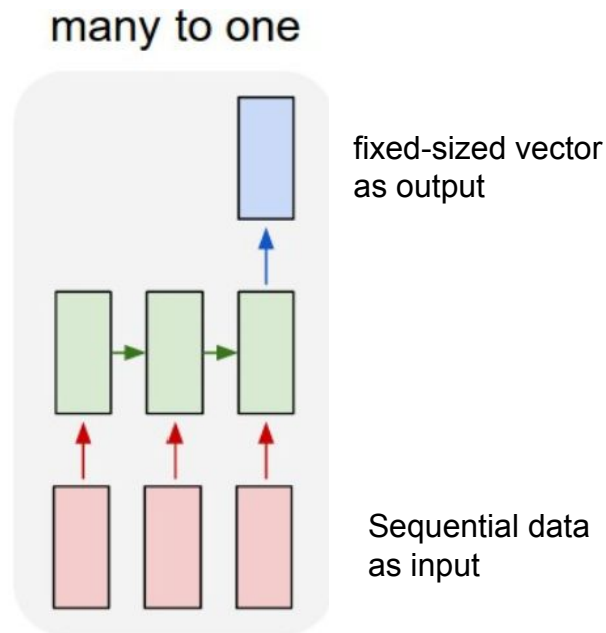
Limitation Vanilla NN:

- accept a **fixed-sized vector as input** (e.g. an image) and produce a **fixed-sized vector as output** (e.g. probabilities of different classes).
- These models perform this mapping using a fixed amount of computational steps (e.g. the number of layers in the model)



Sequences in the input, the output, or in the most general case both

Sequence Modeling: Many to One (Sequences in the **Input**)



Each rectangle is a vector and arrows represent functions (e.g. matrix multiply). Input vectors are in red, output vectors are in blue and green vectors hold the hidden state.

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

The **input** data is a **sequence**, but the output is a fixed-size vector or scalar, not a sequence.

For example, in sentiment analysis, the input is text-based (for example, a movie review) and the output is a class label (for example, a label denoting whether a reviewer liked the movie).

Text Classification /
Regression

Sentiment Analysis

Essay scoring

Time-series Classification /
Regression

Churn detection

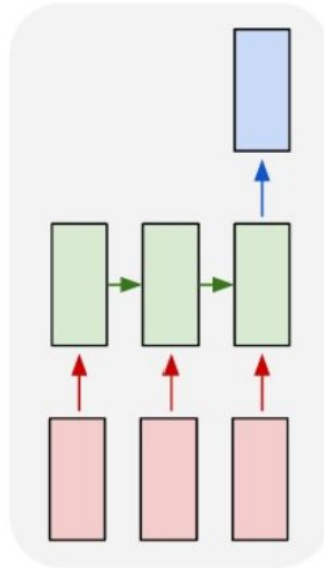
Demand forecasting

Video Classification

Deepfake detection

Sequence Modeling: Many to One

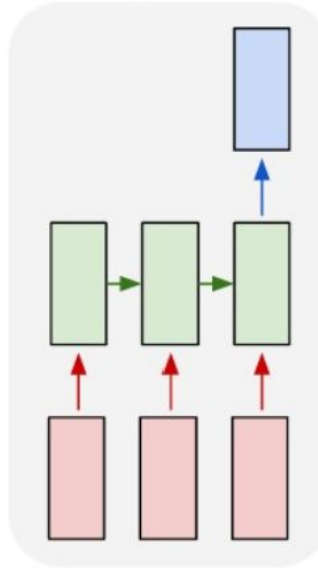
positive



I am happy

Sentiment Analysis

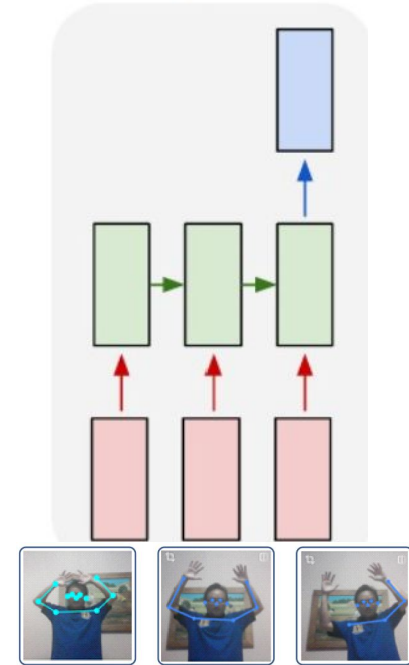
17089



17027 17005 17059
(070426) (080426) (090426)

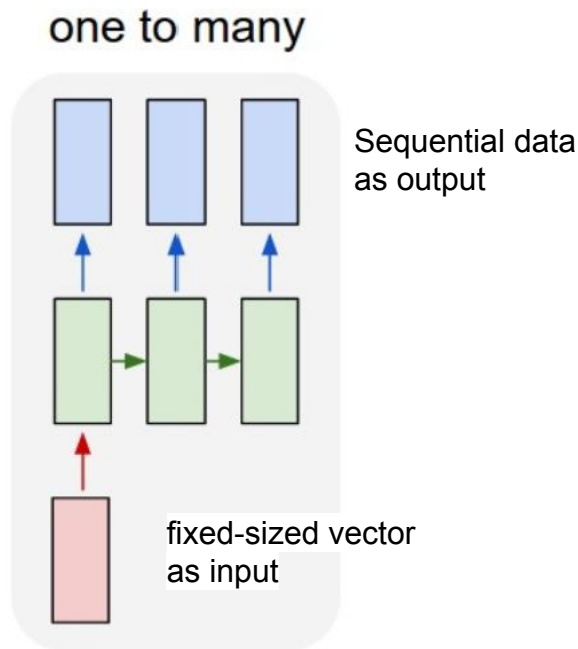
USDToIDR Forecasting

Angkat tangan



Pose classification

Sequence Modeling: One to Many (Sequences in the **Output**)



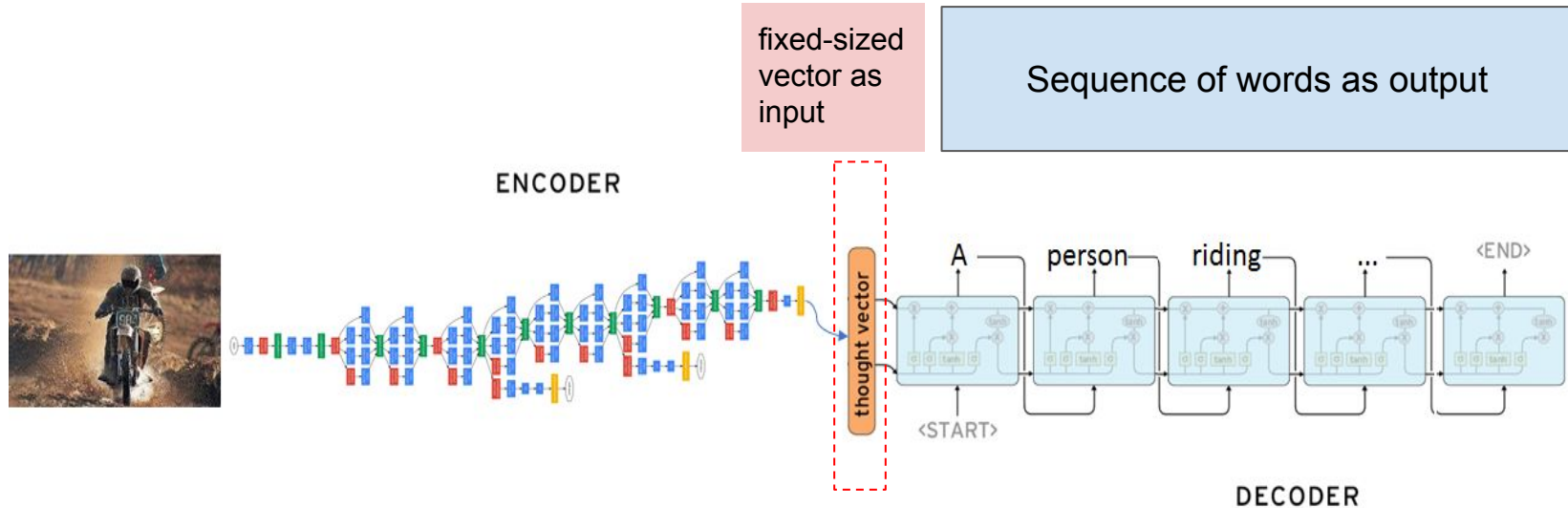
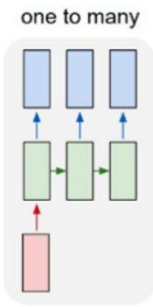
The input data is in standard format and not a sequence, but the output is a sequence.

An example of this category is image captioning—the input is an image and the output is an English phrase summarizing the content of that image.

Each rectangle is a vector and arrows represent functions (e.g. matrix multiply). Input vectors are in red, output vectors are in blue and green vectors hold the hidden state.

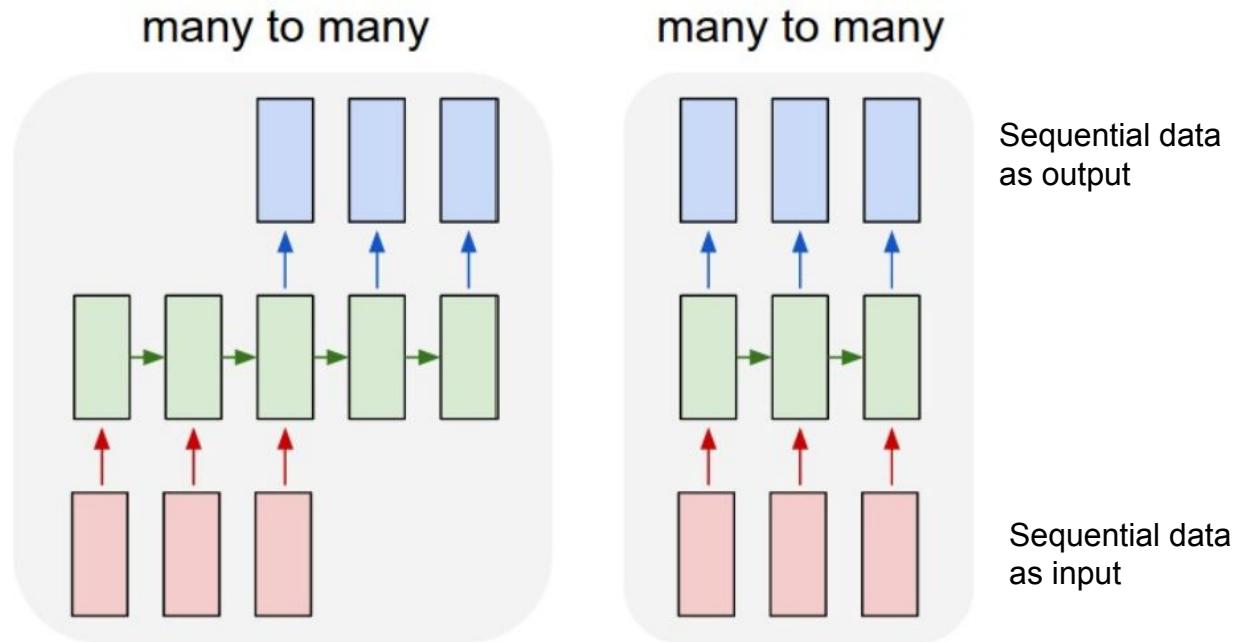
<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Seq. Modeling - One to Many: Image Captioning



CNN *Encoder* (Inception) - RNN *Decoder* (LSTM) (Vinyals dkk., 2014)

Sequence Modeling: Many to Many

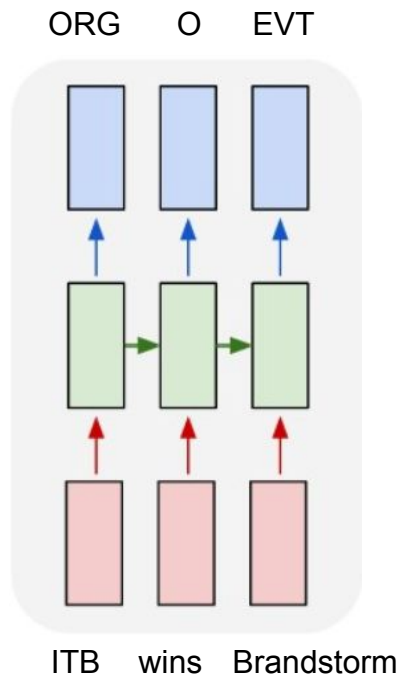


Each rectangle is a vector and arrows represent functions (e.g. matrix multiply). Input vectors are in red, output vectors are in blue and green vectors hold the hidden state.

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

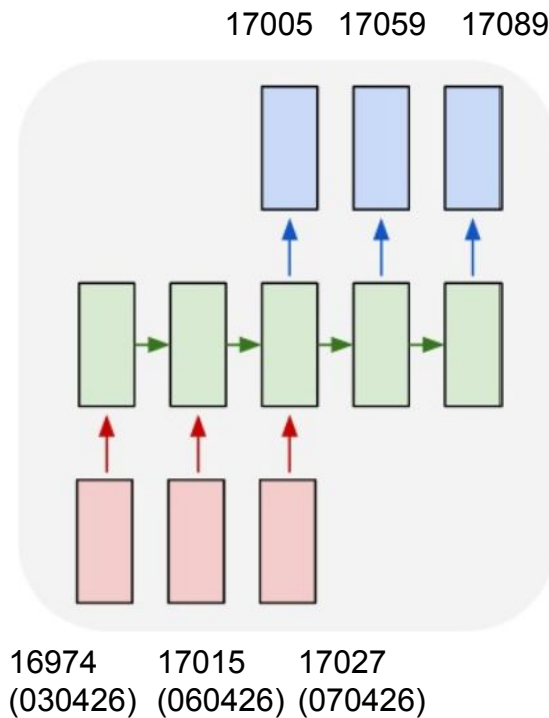
- Both the input and output arrays are sequences.
- This category can be further divided based on whether the input and output are synchronized or delayed.

Sequence Modeling: Many to Many



Synchronized
Many to Many

Delayed
Many to Many



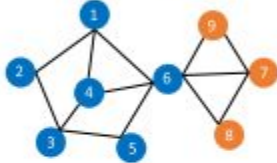
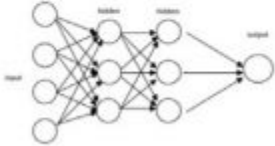
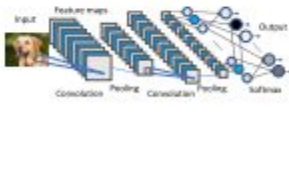
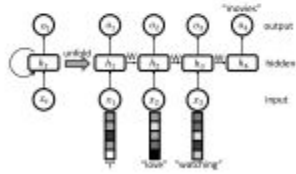
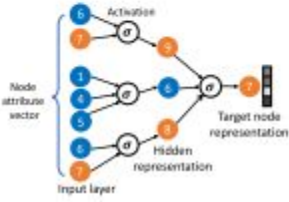
Named-entity Recognition

RNNs for Modeling Sequences

What & Why

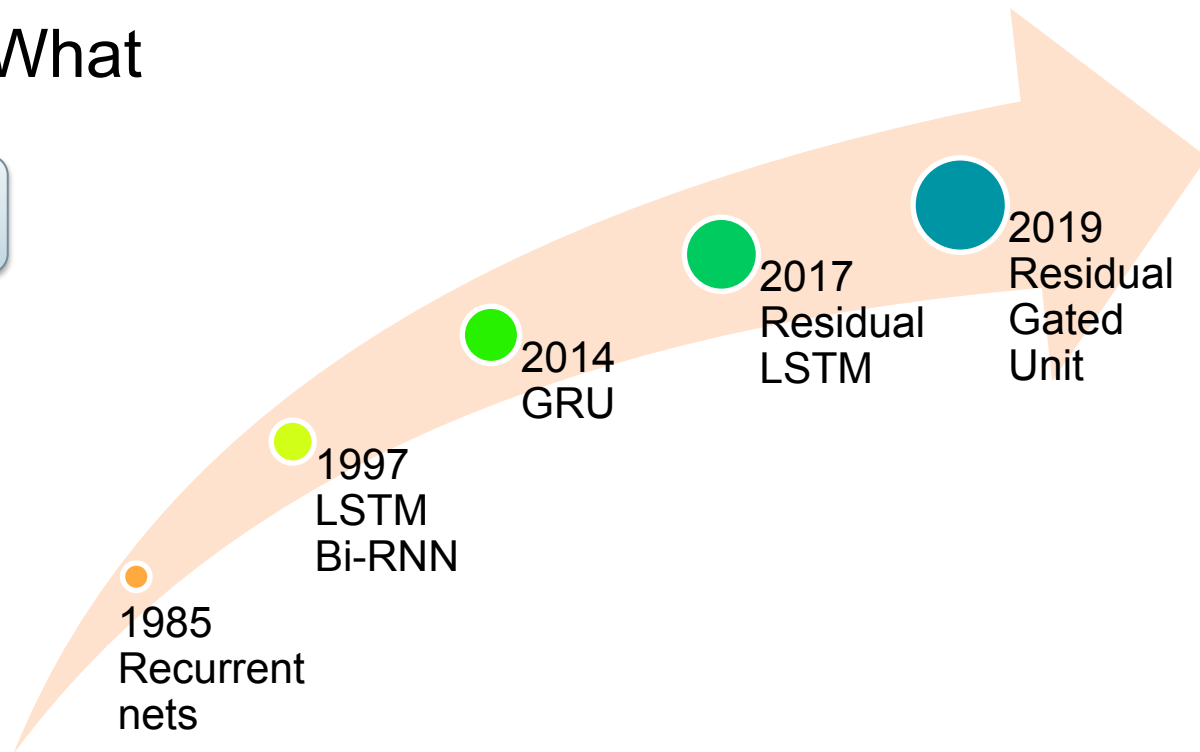
Deep Learning Architecture

Table 10.5 An overview of typical deep learning architectures.

Data Type	Multidimensional	Grid	Sequence	Graph
	Features: credit rating, account balance $x = (4.5, 500, 3, 5)$ #deposits, #withdraws		$x = \text{"I love watching movies."}$	
DL Architecture	Feed-forward Network	CNN	RNN	GNN
				

Recurrent NN: What

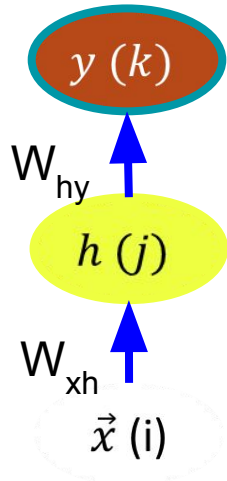
ANN with forward
and backward link



- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1985). *Learning internal representations by error propagation* (No. ICS-8506). California Univ San Diego La Jolla Inst for Cognitive Science.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE transactions on Signal Processing*, 45(11), 2673-2681.
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. In *Advances in neural information processing systems* (pp. 3104-3112).
- Kim, J., El-Khamy, M., & Lee, J. (2017). Residual LSTM: Design of a deep recurrent architecture for distant speech recognition. *arXiv preprint arXiv:1701.03360*.
- Luo, H., Li, T., Liu, B., & Zhang, J. (2019). DOER: Dual cross-shared RNN for aspect term-polarity co-extraction. *arXiv preprint arXiv:1906.01794*.

Dataflow: Feedforward vs Recurrent Neural Network

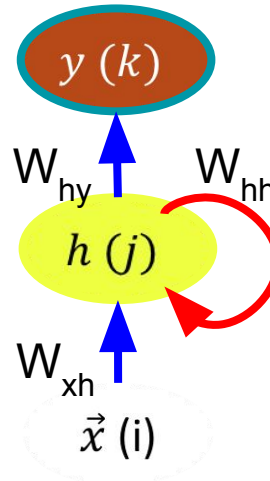
Feedforward NN



Forward link only.
Information flows from the input to the hidden layer, and then from the hidden layer to the output layer.

there isn't any concept of order in time between the data

Recurrent (Feedback) NN



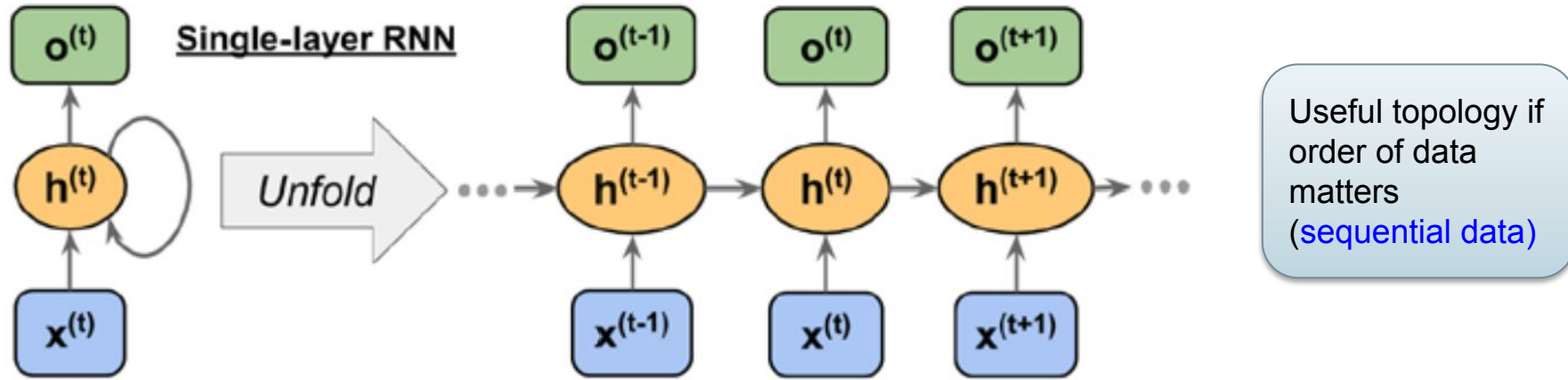
Forward **and backward** link.
*The hidden layer receives its input from **both** the **input layer** of the current time step and the **hidden layer** from the previous time step.*

Useful topology if order of data matters (**sequential data**)

Sharing parameter through sequence / timestep

Notes: Both of these networks have only one hidden layer

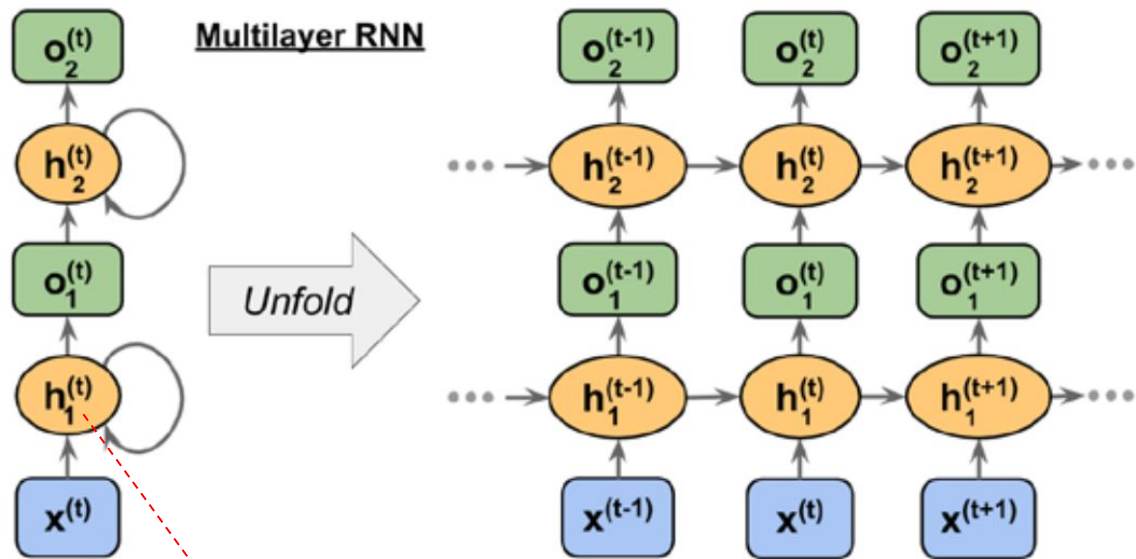
Dataflow of Recurrent Neural Network: Single-layer RNN



To examine the architecture of RNNs and the flow of information, a compact representation with a recurrent edge can be [unfolded](#). Each hidden unit in an RNN receives two distinct sets of input—the preactivation from the input layer and the activation of the same hidden layer from the previous time step, $t - 1$.

- At the first time step, $t = 0$, the hidden units are initialized to zeros or small random values.
- At a time step where $t > 0$, the hidden units receive their input from the data point at the current time, $x^{(t)}$, and the previous values of hidden units at $t - 1$, indicated as $h^{(t-1)}$.

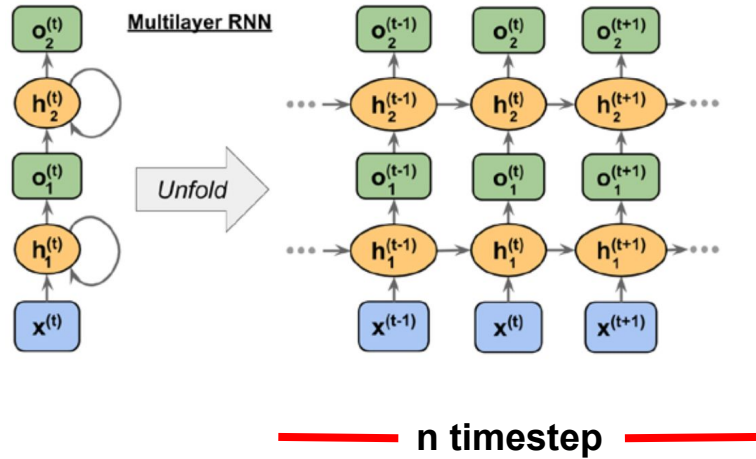
Dataflow of Recurrent Neural Network: Multi-layer RNN



layer = 2: The second hidden layer, $h_2^{(t)}$, receives its inputs from the outputs of the layer below at the current time step ($o_1^{(t)}$) and its own hidden values from the previous time step, $h_2^{(t-1)}$.

layer = 1: The hidden layer is represented as $h_1^{(t)}$ and it receives its input from the data point $x^{(t)}$, and the hidden values in the same layer, but at the previous time step, $h_1^{(t-1)}$.

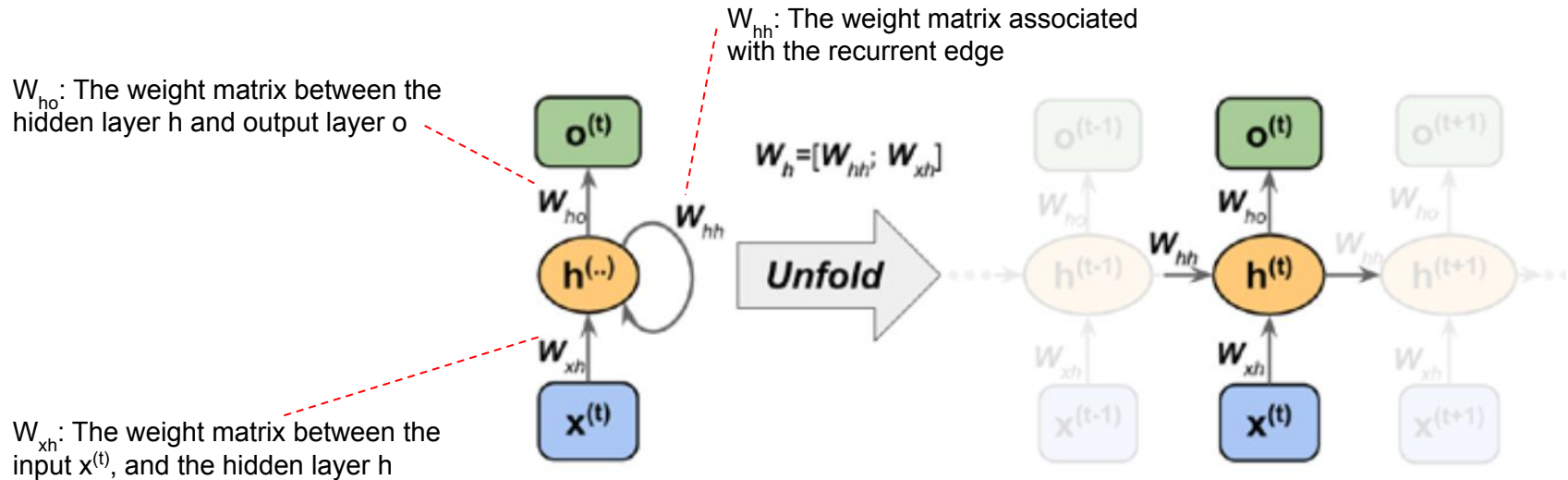
Recurrent Neural Network: Neuron Dependent



- Each recurrent layer must receive a sequence as input,
- All the recurrent layers **except the last one must return** a sequence as output (that is, we will later have to set `return_sequences=True`).
- The behavior of the last recurrent layer depends on the type of problem.

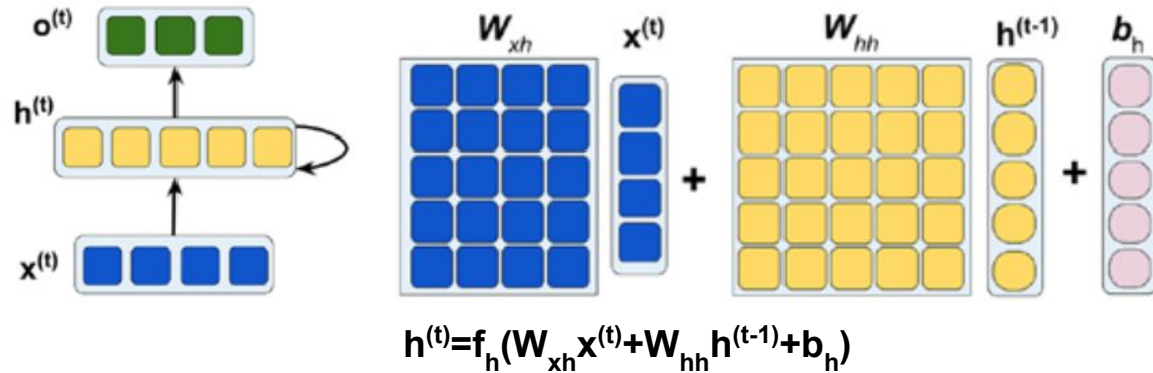
A RNN unit has loops in them that **allow information to be carried across neurons** while reading in input. RNN cannot rely on the input alone and must use its recurrent connection to keep track of the context to achieve this task.

RNN: Weights or Parameters



- Each directed edge (the connections between boxes) in the representation of an RNN is associated with a weight matrix.
- Those weights do not depend on time, t ; therefore, they are **shared across the time axis (Parameter Sharing)**

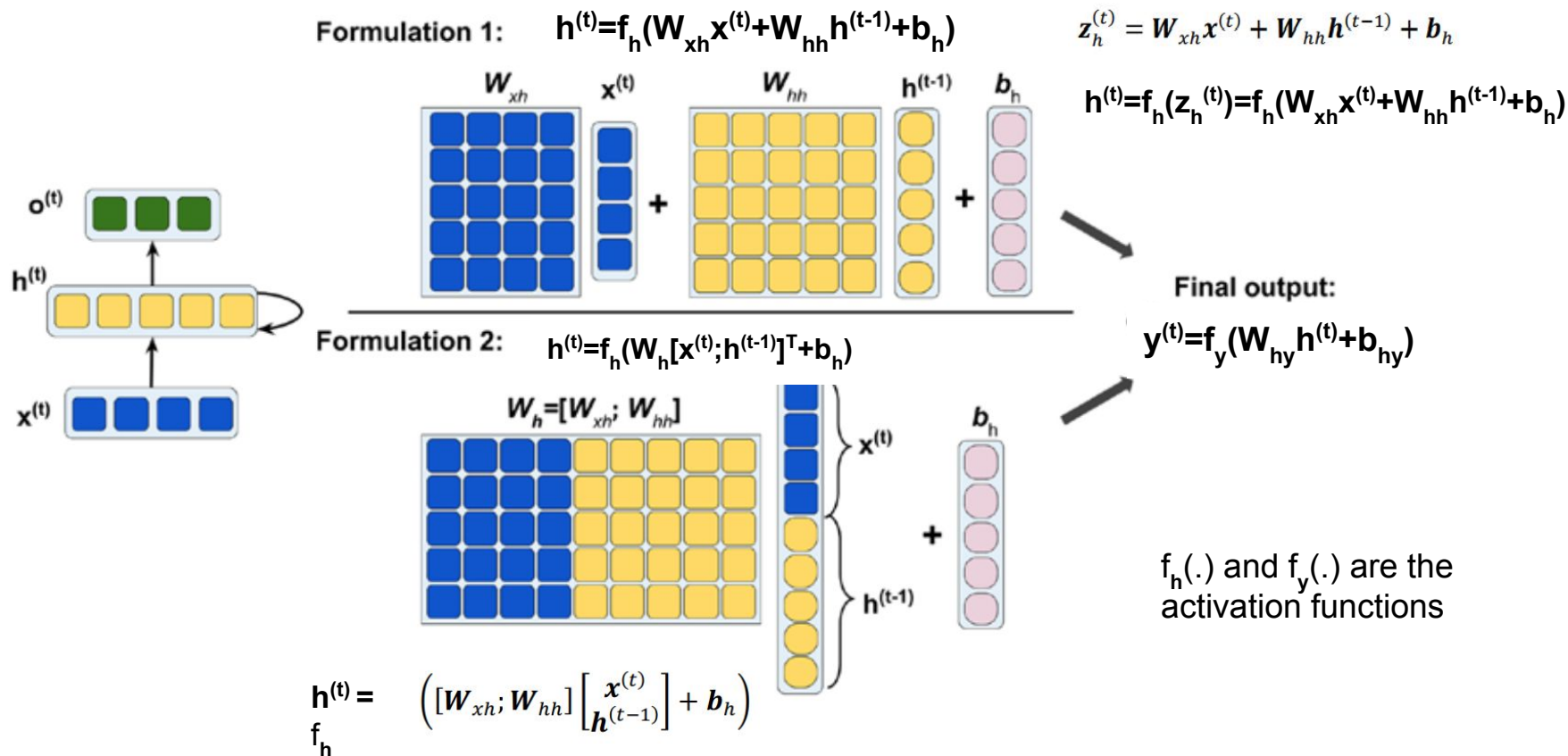
RNN: Weight Matrix Dimension



- W_{xh} : hidden neurons x input dimension $x^{(t)}$
- Bias b_h : hidden neurons x 1
- W_{hh} : hidden neurons x hidden neurons
- W_{hy} : output neurons x hidden neurons
- Bias b_y : output neurons x 1

$f_h(.)$ is the activation function

RNN: Forward Propagation



Dataset Preparation

Prepare Time-series Dataset

The time series data has to be transformed into a matrix in which each value is related to the time window (lags) that precedes it.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Airline Passenger Dataset for Total Passengers Prediction

"Month","Passengers"

"1949-01",112

"1949-02",118

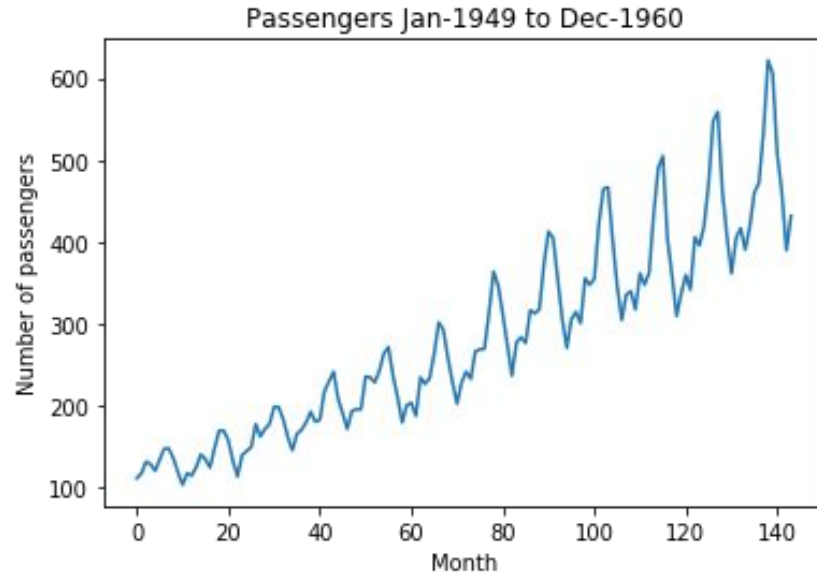
"1949-03",132

"1949-04",129

"1949-05",121

...

It contains 144
observations (total
passengers in thousands



Total Passengers Prediction: FFNN 1 Feature

"Month", "Passengers"

"1949-01", 112

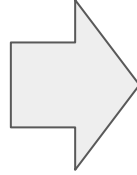
"1949-02", 118

"1949-03", 132

"1949-04", 129

"1949-05", 121

...



1 Feature Dataset:

$X^{(t)}$ $Y=X^{(t+1)}$

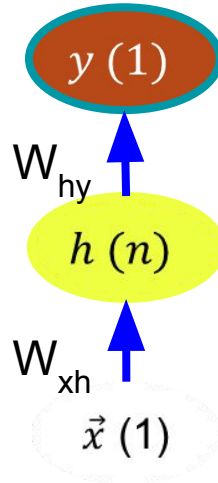
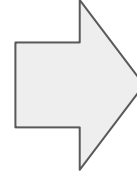
112 118

118 132

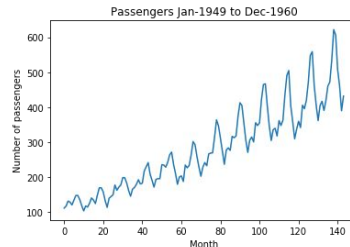
132 129

129 121

121 135



What is the weakness of this model ?



Total Passengers Prediction: FFNN 3 Features

"Month", "Passengers"

"1949-01", 112

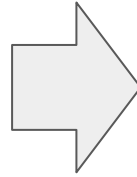
"1949-02", 118

"1949-03", 132

"1949-04", 129

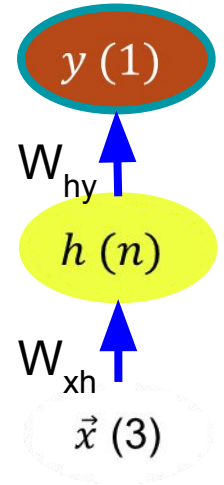
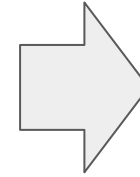
"1949-05", 121

...



3 Feature Dataset:

$X1=X^{(t-2)}$	$X2=X^{(t-1)}$	$X3=X^{(t)}$	$Y=X^{(t+1)}$
112	118	132	129
118	132	129	121
132	129	121	135
129	121	135	148
121	135	148	148



What is the weakness of this model ?

Total Passengers Prediction: RNN Single Layer

"Month","Passengers"

"1949-01",112

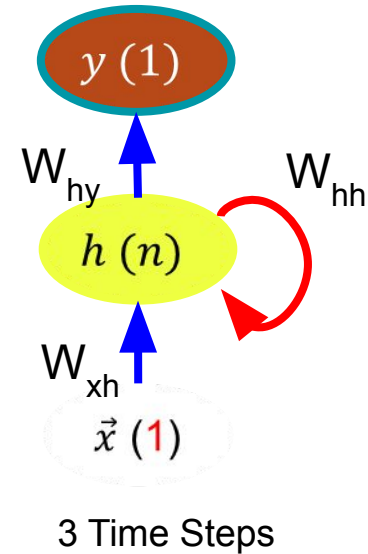
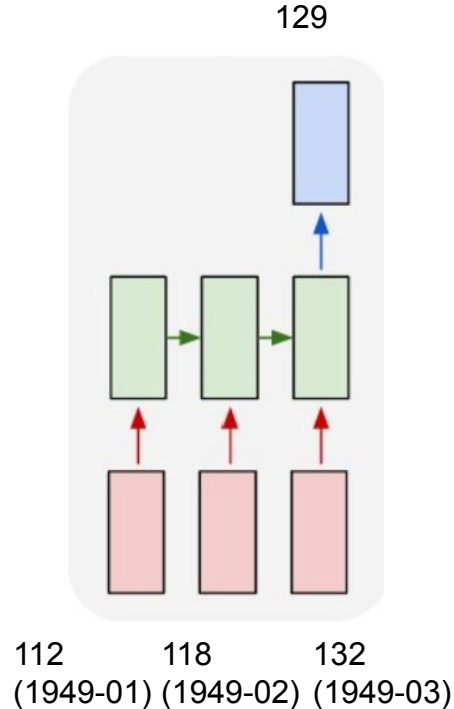
"1949-02",118

"1949-03",132

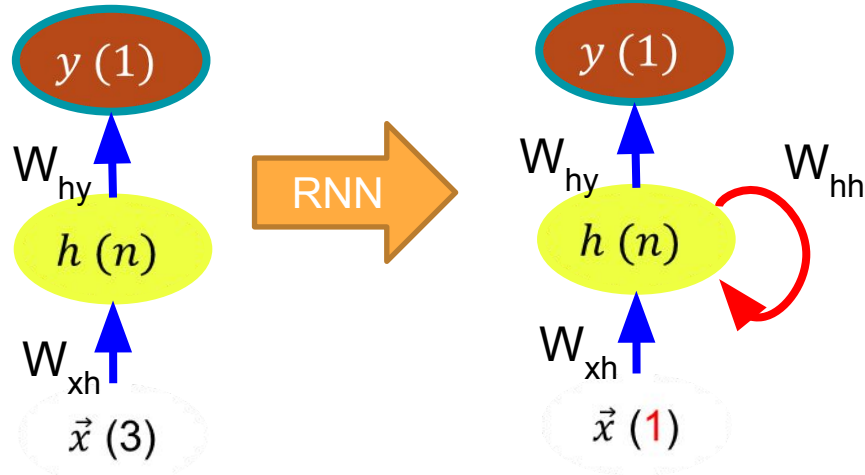
"1949-04",129

"1949-05",121

...



FFNN vs RNN



3 Feature Dataset:

$X1=X^{(t-2)}$	$X2=X^{(t-1)}$	$X3=X^{(t)}$	$Y=X^{(t+1)}$
112	118	132	129
118	132	129	121
132	129	121	135
129	121	135	148
121	135	148	148

1 Feature with 3 time steps Dataset:

X	$Y=X^{(t+1)}$
[112] [118] [132]	129
[118] [132] [129]	121
[132] [129] [121]	135
[129] [121] [135]	148
[121] [135] [148]	148

Forward Propagation: Example

Sequence Data: Example

Given Sequence: ABCCD

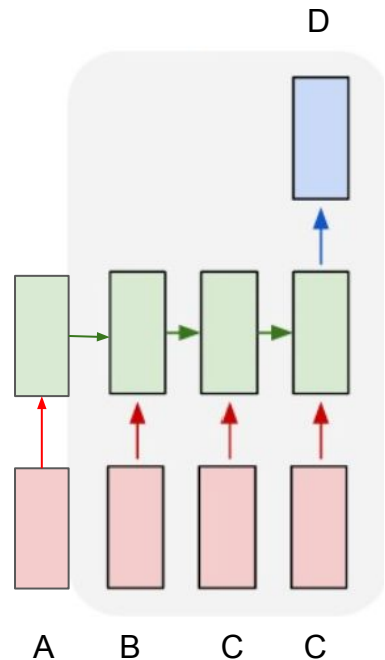
- Browsing history:
{Homepage}{Electronics}{Camera}{Camera}{ShoppingCart}...
A=Homepage; B=Electronics; C=Camera; D=ShoppingCart
- Language model of character: hello...
- Language model of words: aku sedang pura pura tertawa ...
A=aku; B=sedang; C=pura; D=tertawa

Task: Next Item Prediction

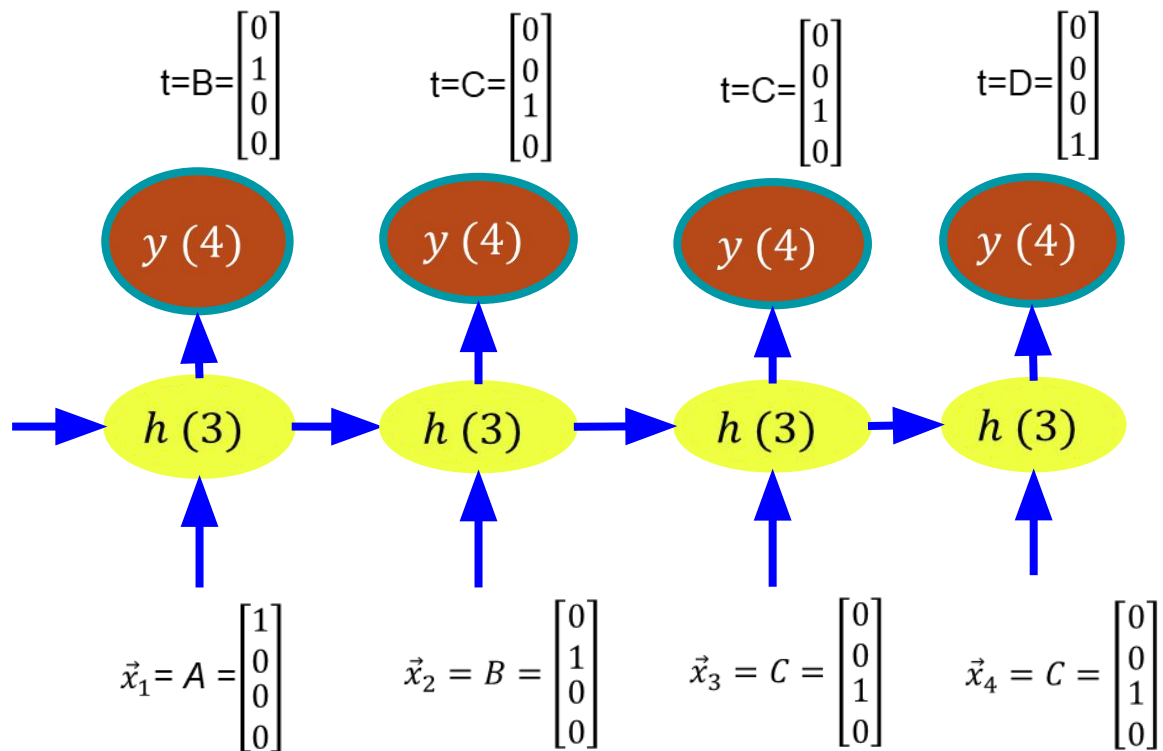
- Length input sequence: 4
- For given sequence ABCCD:
 - Input: ABCC
 - Output: D
- Encode all items into vector, eg. one-hot encoding

Vector representation:

$$A = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, B = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, C = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, D = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \Rightarrow \quad \text{ABCCD} \dots = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \dots$$



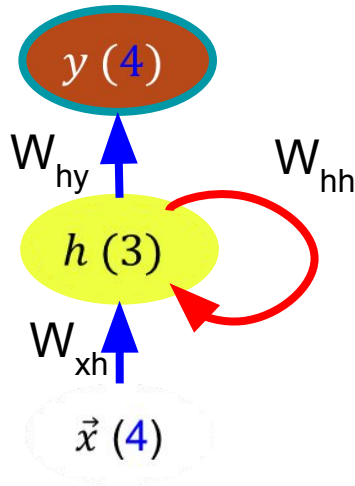
RNN Architecture



Sequence: **A****B****C****C****D**...

A1	A2	A3	A4	Class
1	0	0	0	B
0	1	0	0	C
0	0	1	0	C
0	0	1	0	D
...				

Sequence Classification: RNN



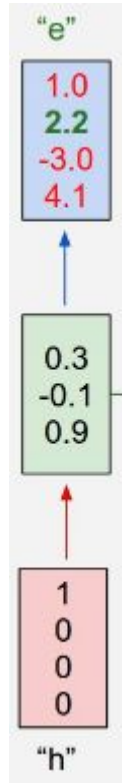
- W_{xh} : matrix 3x4 (hidden neurons x input dimension)
- W_{hh} : matrix 3x3 (hidden neurons x hidden neurons)
- W_{hy} : matrix 4x3 (output neurons x hidden neurons)
- Bias_{xh} : matrix 3x1
- Bias_{hy} : matrix 4x1

$$h^{(t)} = f_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_{xh}) = \tanh(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_{xh})$$

$$y^{(t)} = f_y(W_{hy}h^{(t)} + b_{hy}) = \text{softmax}(W_{hy}h^{(t)} + b_{hy})$$

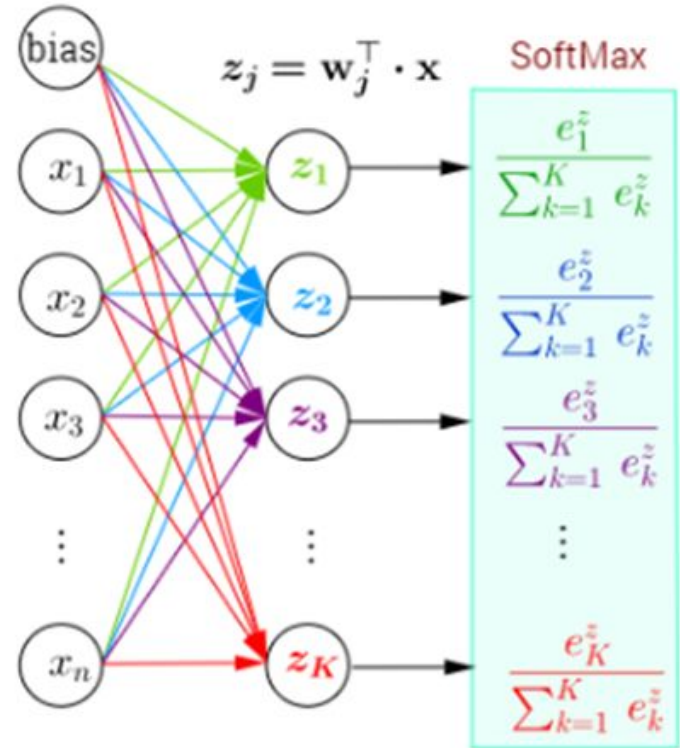
f_h : tanh; f_y : softmax

Output with Softmax

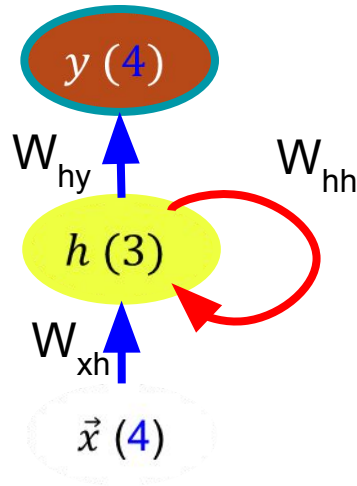


Target: 'e'
Output: 'o'

z	exp(z)	P(z)
1	2,718	0
2,2	9,025	0,1
-3	0,05	0
4,1	60,34	0,8
	72,13	1



Weight Initialization: Example (Random)



W_{xh} 3x4			
0.100	0.150	0.200	0.300
0.150	0.200	0.300	0.100
0.200	0.300	0.100	0.150

W_{hh} 3x3		
0.500	0.500	0.500
0.500	0.500	0.500
0.500	0.500	0.500

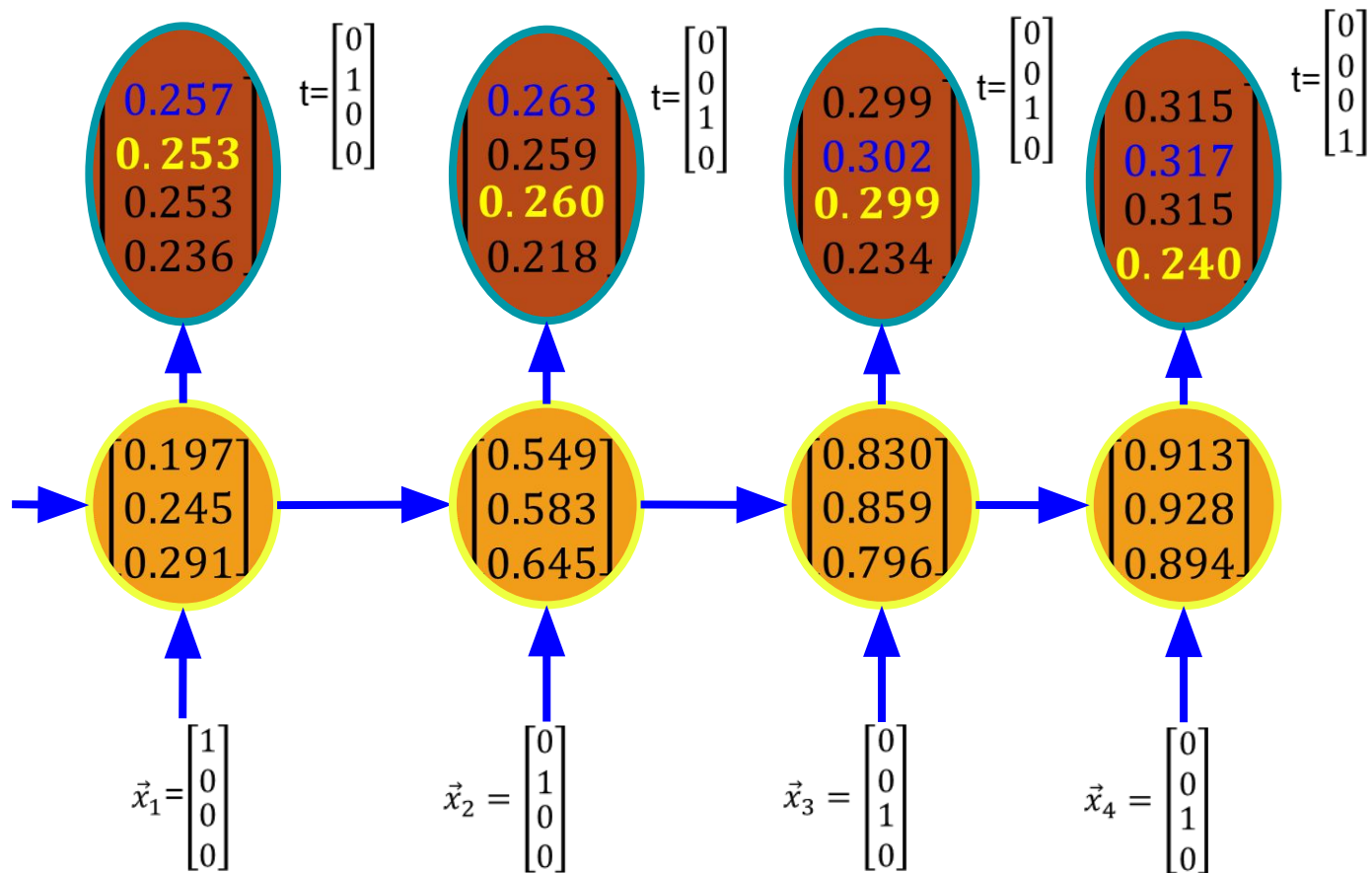
b_{xh}	h_0
0.100	0
0.100	0
0.100	0

W_{hy} 4x3		
0.100	0.200	0.300
0.200	0.300	0.100
0.300	0.100	0.200
0.100	0.100	0.100

b_{hy}
0.100
0.100
0.100
0.100

At the first time step, $t = 0$, the hidden units are initialized to zeros or small random values.

Forward Propagation: 1 instance, 4 timestep



Computing h_t and y_t : Timestep t1 and t2

$$h^{(t)} = f_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_{xh}) = \tanh(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_{xh})$$

$$y^{(t)} = f_y(W_{hy}h^{(t)} + b_{hy}) = \text{softmax}(W_{hy}h^{(t)} + b_{hy})$$

$$t1 = \left\langle \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \right\rangle$$

$W_{xh}x^{(t)}$	$W_{hh}h^{(t-1)} + b_{xh}$	net_h ^(t)	$h^{(t)}$
0.100	0.100	0.200	0.197
0.150	0.100	0.250	0.245
0.200	0.100	0.300	0.291

$W_{hy}h^{(t)} + b_{hy}$	$\exp(W_{hy}h^{(t)} + b_{hy})$	$y^{(t)}$
0.256	1.292	0.257
0.242	1.274	0.253
0.242	1.274	0.253
0.173	1.189	0.236

$$t2 = \left\langle \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\rangle$$

$W_{xh}x^{(t)}$	$W_{hh}h^{(t-1)} + b_{xh}$	net_h ^(t)	$h^{(t)}$
0.150	0.467	0.617	0.549
0.200	0.467	0.667	0.583
0.300	0.467	0.767	0.645

$W_{hy}h^{(t)} + b_{hy}$	$\exp(W_{hy}h^{(t)} + b_{hy})$	$y^{(t)}$
0.465	1.592	0.263
0.449	1.567	0.259
0.452	1.571	0.260
0.278	1.320	0.218

Computing h_t and y_t : Timestep t3 and t4

$$h^{(t)} = f_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_{xh}) = \tanh(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_{xh})$$

$$y^{(t)} = f_y(W_{hy}h^{(t)} + b_{hy}) = \text{softmax}(W_{hy}h^{(t)} + b_{hy})$$

$$t3 = \left\langle \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\rangle$$

$W_{xh}x^{(t)}$	$W_{hh}h^{(t-1)} + b_{xh}$	net_h ^(t)	$h^{(t)}$
0.200	0.988	1.188	0.830
0.300	0.988	1.288	0.859
0.100	0.988	1.088	0.796

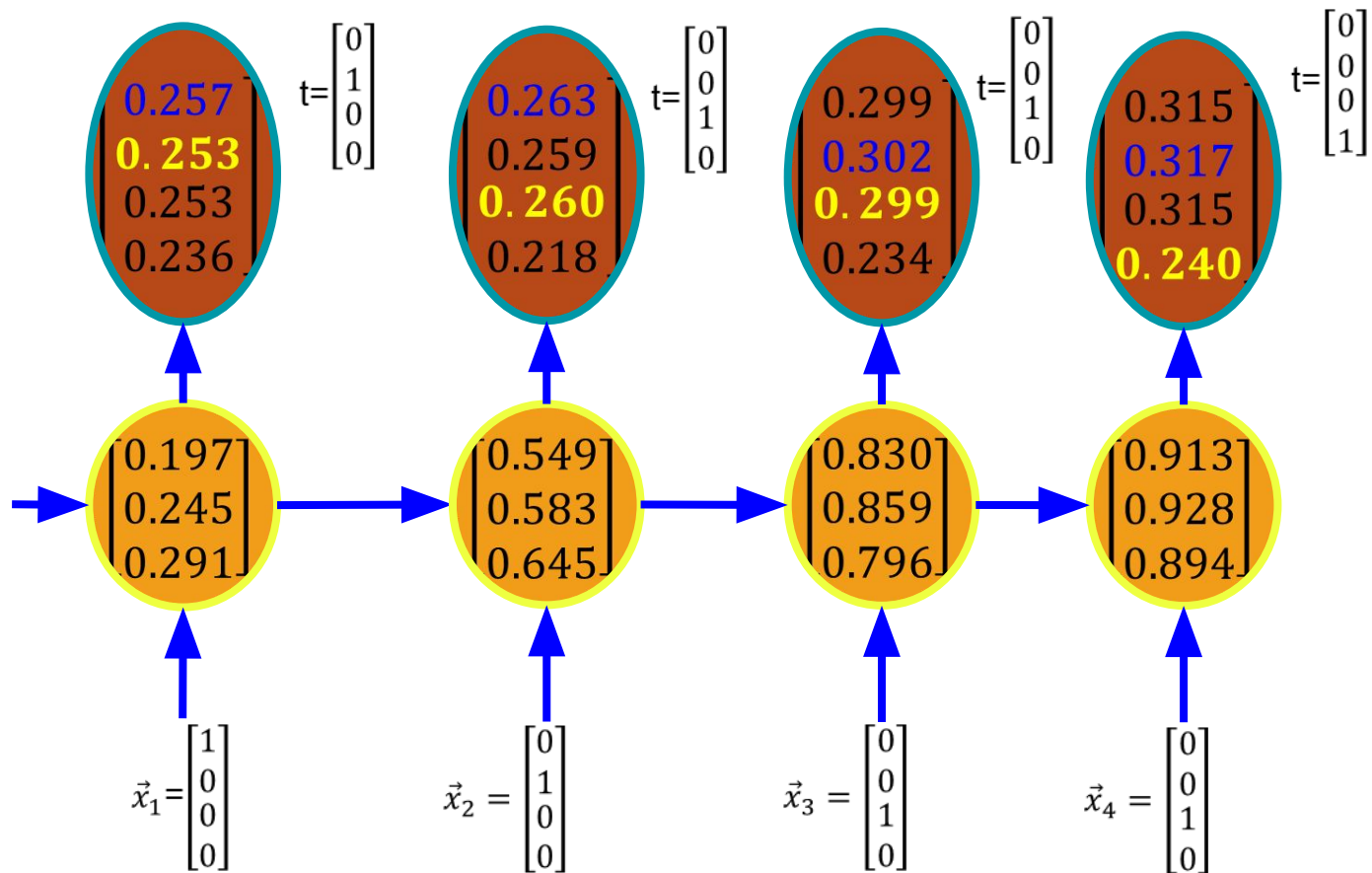
$W_{hy}h^{(t)} + b_{hy}$	$\exp(W_{hy}h^{(t)} + b_{hy})$	$y^{(t)}$
0.594	1.811	0.299
0.603	1.828	0.302
0.594	1.812	0.299
0.349	1.417	0.234

$$t4 = \left\langle \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\rangle$$

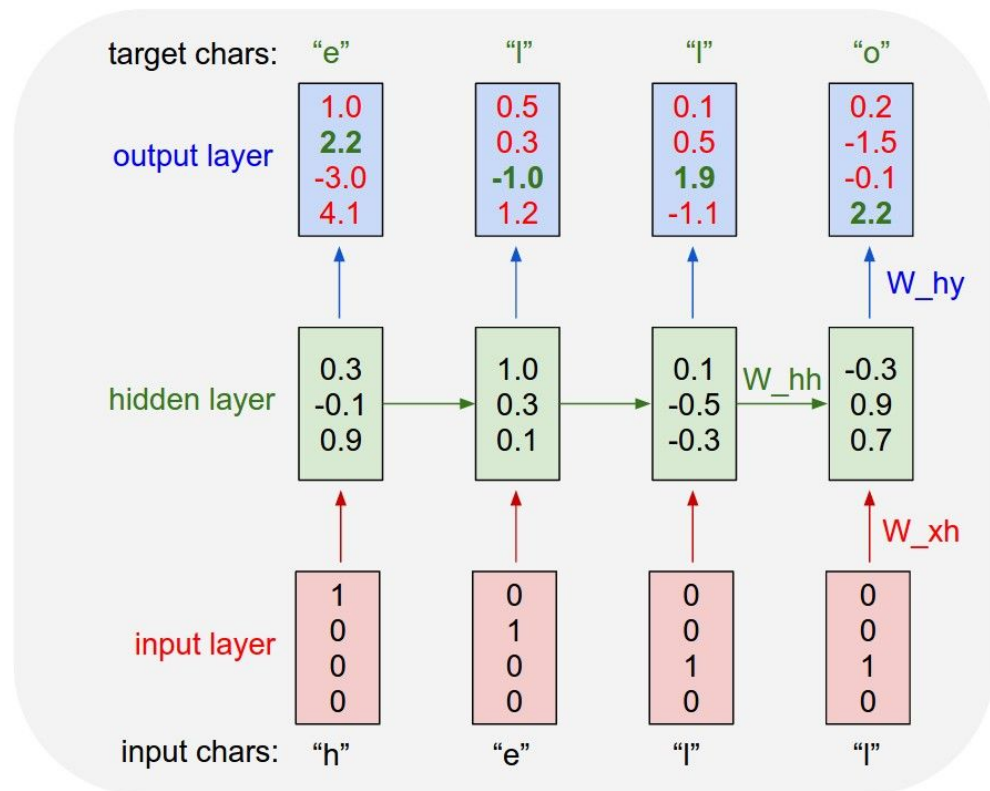
$W_{xh}x^{(t)}$	$W_{hh}h^{(t-1)} + b_{xh}$	net_h ^(t)	$h^{(t)}$
0.200	1.343	1.543	0.913
0.300	1.343	1.643	0.928
0.100	1.343	1.443	0.894

$W_{hy}h^{(t)} + b_{hy}$	$\exp(W_{hy}h^{(t)} + b_{hy})$	$y^{(t)}$
0.645	1.906	0.315
0.650	1.916	0.317
0.645	1.907	0.315
0.373	1.453	0.240

Forward Propagation: 1 instance, 4 timestep



Language Model of Character

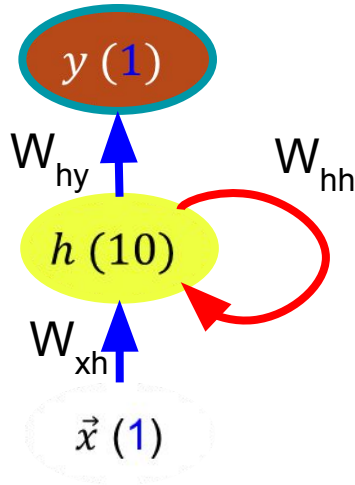


- t=1 (input "h"): output="o", target="e"
- t=2 (input "e"): output="o", target="l"
- t=3 (input "l"): output="l", target="l"
- t=4 (input "l"): output="o", target="o"

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

RNN Implementation

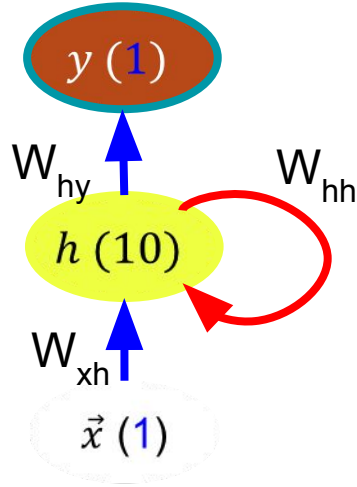
Implementing RNN on Keras: Many to One



predict amazon stock closing prices, RNN 50 timestep

```
from keras import Sequential
from keras.layers import SimpleRNN, Dense
model = Sequential()
model.add(SimpleRNN(10, input_shape=(50,1)))
#simple recurrent layer, 10 neurons & process 50x1 sequences
model.add(Dense(1,activation='linear'))
#linear output because this is a regression problem
```

Number of Parameter



Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
simple_rnn (SimpleRNN)	(None, 10)	120
=====		
dense (Dense)	(None, 1)	11
=====		
Total params: 131		
Trainable params: 131		
Non-trainable params: 0		

$$\text{Total parameter} = (1+10+1)*10+(10+1)*1=131$$

Simple RNN:

W_{xh} : matrix hidden neurons x (input dimension + 1)

W_{hh} : matrix hidden neurons x hidden neurons

W_{hy} : matrix output neurons x (hidden neurons+1)

Number of Parameter: Example 2

```
model = Sequential() #initialize model
model.add(SimpleRNN(64, input_shape=(50,1), return_sequences=True)) #64 neurons
model.add(SimpleRNN(32, return_sequences=True)) #32 neurons
model.add(SimpleRNN(16)) #16 neurons
model.add(Dense(8, activation='tanh'))
model.add(Dense(1, activation='linear'))
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 50, 64)	4224
simple_rnn_2 (SimpleRNN)	(None, 50, 32)	3104
simple_rnn_3 (SimpleRNN)	(None, 16)	784
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 1)	9

Total params: 8,257

Trainable params: 8,257

Non-trainable params: 0

$$= (1+64+1)*64=4224$$

$$= (64+32+1)*32=3104$$

$$= (32+16+1)*16=784$$

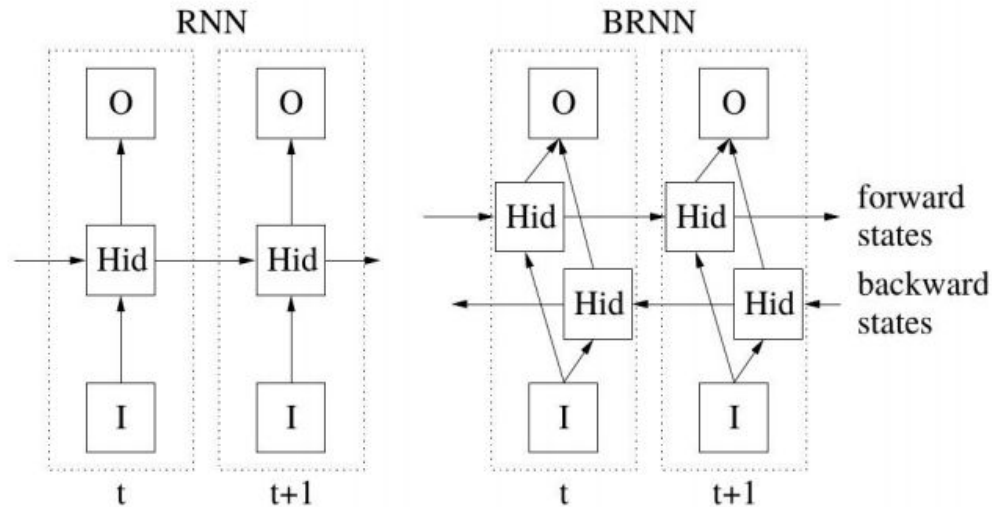
$$= (16+1)*8=136$$

$$= (8+1)*1=9$$

Total parameter = 8257

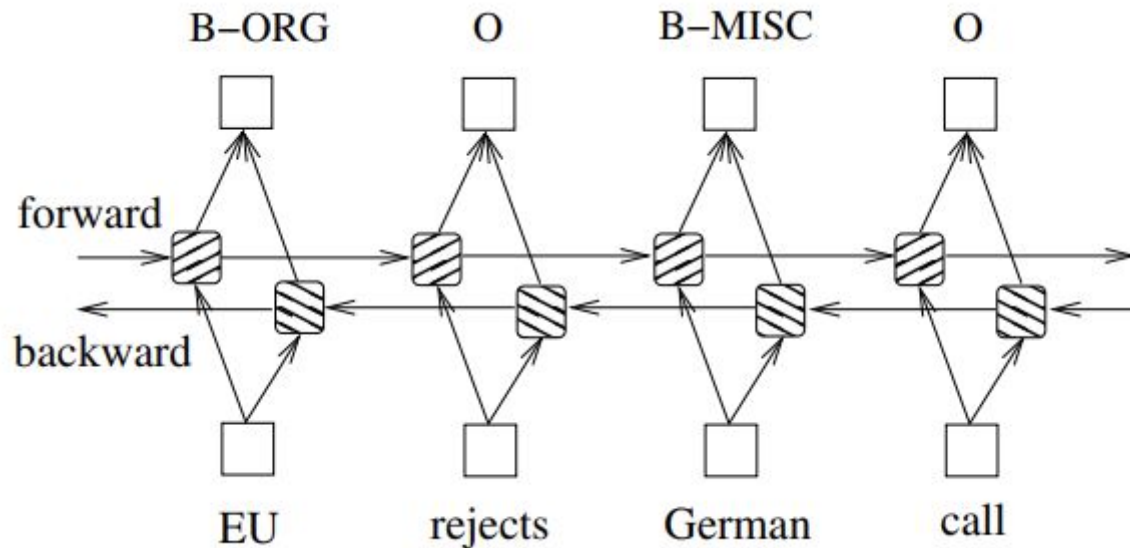
Bidirectional RNNs

In many applications we want to output a prediction of $y(t)$ which may depend on the whole input sequence. E.g. co-articulation in speech recognition, right neighbors in POS tagging, etc. **Bidirectional RNNs** combine an RNN that moves forward through time beginning from the start of the sequence with another RNN that moves backward through time beginning from the end of the sequence.



https://www.cs.toronto.edu/~tingwuwang/rnn_tutorial.pdf

Bidirectional RNNs for Information Extraction



<https://www.depends-on-the-definition.com/sequence-tagging-lstm-crf/>

Latihan RNN

[Share Sub Module](#)

Latihan RNN

Type: **Quiz**

⌚ Start: 13 April 2026 12:00

⌚ End: 23 April 2026 23:00

Passing Grade: **Not Set**

Duration: **15060**
Minutes

✓ Show Score

✓ Show Solution

✓ Can Go Back

You have **1** packages

[Open Quiz](#)[Results](#)

Next

